CULINARY RESTAURANT

Informe de avance: integraciones preparadas para producción

Qué se integró, cómo se activa cada conexión real y qué hay que conseguir con cada proveedor (cuentas, credenciales y configuración), paso a paso.

Implementado y probado con proveedores simulados

Pendiente: activar con cuentas reales

FECHA
23 de octubre de 2026

ENTREGA
PR #4 · rama integraciones-produccion

VERSIÓN
Culinary Restaurant 0.3.0

1. Resumen ejecutivo

Las integraciones con terceros que hasta ahora sólo existían como simuladores quedaron programadas contra las API reales de cada proveedor y listas para activarse. Ninguna se conecta sola: cada una necesita su archivo de credenciales en el servidor de la sucursal y que el gerente la cambie a modo «Real».

13	598/598	20/20	52
conexiones con proveedores	comprobaciones de integración	pruebas de interfaz (Edge)	pruebas nuevas sin red

Área	Proveedor	Qué hace en modo real
Facturación CFDI 4.0	Facturama (sandbox o producción)	Timbra, consulta y cancela facturas; opcionalmente envía PDF y XML al cliente. Nunca timbra dos veces.
Terminal bancaria	Mercado Pago Point · Clip · BBVA	Envía el cobro a la terminal y sigue su resultado en vivo; cancela y devuelve. BBVA se registra con el voucher.
Pagos en línea	Stripe Checkout	Crea ligas de pago, las manda por WhatsApp, aplica el pago y reembolsa si la cuenta ya estaba pagada.
Mensajería	WhatsApp Cloud API (Meta) · correo SMTP	Avisos de pedido en camino, listo para recoger, ligas de pago y avisos al dueño.
Plataformas de reparto	Uber Eats · Rappi · DiDi Food vía Deliverect	Recibe pedidos, los manda a cocina, devuelve los estados y publica el menú.
Hotel	Cloudbeds · Oracle OPERA (OHIP)	Valida al huésped y su crédito y carga el consumo a la habitación.
Mapas	Google Geocoding y Routes	Ubica direcciones de entrega y calcula distancia y tiempo.
Identificador de llamadas	Conmutador Asterisk	Muestra quién llama y lo liga a su ficha de cliente.
Impresora y cajón	ESC/POS (red o USB)	Imprime cuentas y cortes, abre el cajón al cobrar en efectivo.

> **Estado**
> Todo el código está terminado, revisado y probado contra versiones simuladas de cada API (sin red). Lo único pendiente es **conseguir las cuentas y credenciales reales** y hacer la prueba final con ellas. Este documento explica exactamente qué conseguir y cómo.

Contenido

1. Resumen ejecutivo
2. Cómo se activa una integración

3. Garantías de seguridad y dinero
4. Facturama (facturación)
5. Mercado Pago Point
6. Clip
7. BBVA
8. Stripe (ligas de pago)
9. WhatsApp Cloud API
10. Correo SMTP
11. Deliverect (plataformas)
12. Cloudbeds (hotel)
13. Oracle OPERA · OHIP (hotel)
14. Google Maps
15. Asterisk (llamadas)
16. Impresora y cajón
17. Central publicada en internet
18. Lista de verificación final
19. Puntos por confirmar

2. Cómo se activa una integración

Cada integración tiene tres modos en **Restaurante → Integraciones**: Apagada Simulación Real. El botón «Real» sólo se habilita cuando el servidor tiene credenciales válidas; mientras tanto la tarjeta dice Sin credenciales.

1. **Conseguir las credenciales** del proveedor (secciones 4 a 16 de este informe).
2. **Copiar el archivo de ejemplo** que corresponde desde `deploy/integrations/` y llenarlo con los datos reales.
3. **Instalarlo en el servidor de la sucursal** (Debian / CulinaryOS) con dueño `root`, grupo `culinaryos-app` y permisos `0640`, para que sólo el backend lo pueda leer:

```
sudo install -o root -g culinaryos-app -m 0640 terminal.json
/etc/culinaryos/integrations/terminal.json
sudo systemctl restart culinaryos-backend
```

4. **Revisar la tarjeta** en Integraciones: debe decir Credenciales listas. Si dice error: …, el mensaje indica qué campo corregir (un archivo con errores nunca detiene el sistema).
5. **Elegir «Real»**. Desde ese momento la integración contacta al proveedor de verdad.

Integración	Archivo en el servidor	Ejemplo en el repositorio
Facturación	`/etc/culinaryos/integrations/pac.json`	`deploy/integrations/pac.example.json`
Terminal bancaria	`…/integrations/terminal.json`	`terminal.example.json`
Pagos en línea	`…/integrations/onlinepay.json`	`onlinepay.example.json`
WhatsApp y correo	`…/integrations/messaging.json`	`messaging.example.json`
Plataformas (sucursal)	`…/integrations/aggregators.json`	`aggregators.example.json`
Plataformas (portal)	archivo indicado con `--deliverect-config`	`deliverect.central.example.json`
Hotel	`…/integrations/pms.json`	`pms.example.json`
Mapas	`…/integrations/maps.json`	`maps.example.json`
Llamadas	`…/integrations/callerid.json`	`callerid.example.json`
Impresora	`…/integrations/printer.json`	`printer.example.json`

Los archivos anteriores `/etc/culinaryos/fiscal.json` y `/etc/culinaryos/printer.json` siguen funcionando. El instalador de CulinaryOS ya crea la carpeta `/etc/culinaryos/integrations` con permisos `0750`.

3. Garantías de seguridad y dinero

- **Nunca se cobra, factura o publica dos veces.** Cada operación se envía una sola vez con una llave única. Si se pierde la respuesta, el sistema primero pregunta al proveedor qué pasó; si no hay forma de saberlo, la deja «sin respuesta» para que una persona decida con el reporte de la terminal o el portal del proveedor.

- **El dinero sólo se registra cuando el proveedor confirma.** Un pago que llega a una cuenta que ya estaba pagada se devuelve automáticamente (Mercado Pago, Stripe) o se marca para devolverlo a mano (Clip, BBVA, hotel).
- **Un cobro en curso bloquea otro** en la misma cuenta hasta tener su resultado.
- **Los secretos sólo viven en el servidor**, en archivos que sólo lee el backend. Ni las tabletas, ni la base de datos de la sucursal, ni la nube los reciben. Nunca entran datos de tarjeta: sólo marca, últimos 4 dígitos y autorización.
- **Se puede volver atrás** en cualquier momento: cambiar la tarjeta a «Simulación» o «Apagada» detiene las conexiones reales.

4 Facturama · facturación CFDI 4.0

PAC autorizado por el SAT · archivo `pac.json`

Qué quedó integrado: timbrado de solicitudes fiscales en el ambiente de pruebas (sandbox) o en producción, consulta, cancelación con motivo SAT, envío automático del PDF y XML al correo del cliente y botón «Reintentar» para envíos sin respuesta. Antes ya existía la conexión al sandbox; ahora también producción.

Qué se necesita

- Cuenta de Facturama con **plan API** (incluye la API Web) y **folios de API**.
- **Certificado de Sello Digital (CSD)** del RFC emisor: archivos `.cer`, `.key` y su contraseña.
- Perfil fiscal: RFC, razón social, régimen fiscal y código postal del lugar de expedición.
- Claves SAT revisadas con el contador (producto, unidad, objeto de impuesto, forma de pago).
- Para probar primero: una cuenta del **sandbox** de Facturama.

Costo de referencia

Plan anual de API alrededor de **$1,650 MXN** más la compra de folios de API (confirmar precios vigentes en facturama.mx/api-facturacion-electronica). El sandbox es gratuito.

Dónde

https://facturama.mx (producción) · https://apisandbox.facturama.mx (pruebas, con guías en apisandbox.facturama.mx/guias).

Paso a paso

1. **Sandbox (prueba sin validez fiscal).** Crea una cuenta en apisandbox.facturama.mx. Configura el perfil fiscal de pruebas con el RFC `EKU9003173C9` y carga los sellos de prueba que indica la guía del sandbox. El usuario y la contraseña de esa cuenta son los de `pac.json` con `"environment": "sandbox"`.
2. **Obtener el CSD** (si aún no lo tienen). Con la **e.firma** vigente del emisor, en el portal del SAT (sat.gob.mx → Trámites → Certificado de Sello Digital) se genera la solicitud con el programa **Certifica** y se envía con CertiSAT Web. Se descargan el `.cer` y se conserva el `.key` con su contraseña.
3. **Contratar producción.** En facturama.mx crea la cuenta del restaurante, contrata el plan API y compra folios de API.
4. **Configurar el emisor.** En el panel de Facturama → **Configuración / Perfil fiscal**: RFC, razón social, régimen y código postal. En **Sellos digitales (CSD)** sube el `.cer`, el `.key` y la contraseña.
5. **Revisar claves con el contador:** para restaurante normalmente producto `90101501`, unidad `E48`, objeto de impuesto `02`. Se pueden poner claves distintas por producto.
6. **Llenar** `pac.json` con el usuario y contraseña de la cuenta, cambiar a `"environment": "production"` y elegir «Real» en la tarjeta de Facturación.

```
{
  "provider": "facturama",
  "environment": "production",          // "sandbox" para pruebas
  "username": "USUARIO_DE_FACTURAMA",
  "password": "CONTRASENA_DE_FACTURAMA",
  "emailReceiver": true,                // Facturama envía PDF y XML al cliente
  "issuer": {
    "expeditionPlace": "06600",         // CP del lugar de expedición
    "product": { "productCode": "90101501", "unitCode": "E48", "taxObject": "02" },
    "paymentForms": { "card": "04" }
  }
}
```

Importante

En producción cada timbre y cada cancelación tienen validez ante el SAT. Prueben primero con `"sandbox"` y revisen el CFDI generado con el contador antes de cambiar a producción.

5 Mercado Pago Point · terminal

Terminales Point Smart · archivo `terminal.json`

Qué quedó integrado: el cobro se envía desde la cuenta o el Mostrador a la terminal elegida (API de órdenes de Mercado Pago), se sigue cada 3 segundos, se puede cancelar mientras el cliente no ha pagado y se devuelve automáticamente si la cuenta ya estaba cubierta. Una respuesta perdida se recupera sin crear un segundo cobro.

Qué se necesita

- Cuenta de **Mercado Pago** del negocio, verificada, dueña de las terminales.
- Terminales **Point Smart 1 o Point Smart 2** con la versión de software más reciente, vinculadas a esa cuenta.
- Una **aplicación** creada en el panel de desarrolladores y su **Access Token de producción** (empieza con `APP_USR-`).
- Cada terminal en **modo PDV** (punto de venta) y su **id de terminal**.

Dónde

Panel de desarrolladores: mercadopago.com.mx/developers/panel/app.
Documentación: mercadopago.com.mx/developers/es/docs/mp-point.

Costo

La comisión por cobro con terminal de la cuenta Mercado Pago; la API no tiene costo adicional.

Paso a paso

1. Entra con la cuenta del negocio a mercadopago.com.mx/developers/panel/app y pulsa **Crear aplicación**. Ponle nombre (por ejemplo «Culinary Restaurant»), elige la solución de **pagos presenciales / Mercado Pago Point** y acepta los términos.
2. Dentro de la aplicación abre **Credenciales de producción**, completa los datos que pide para activarlas (giro y sitio del negocio) y copia el **Access Token**. Para pruebas existe también un token de prueba con usuarios de prueba.
3. En cada terminal, inicia sesión con la misma cuenta de Mercado Pago (queda vinculada) y actualízala a la última versión.
4. Pon cada terminal en **modo PDV** siguiendo la guía «Configurar terminal» de la documentación de Point (en modo PDV la terminal recibe los cobros que manda el sistema).
5. Obtén el **id de cada terminal** con el token (el id termina con el número de serie de la etiqueta trasera):

```
curl -H "Authorization: Bearer APP_USR-TU_TOKEN" https://api.mercadopago.com/terminals/v1/list
```

6. Agrega cada terminal a `terminal.json` con un nombre fácil para el mesero.

```
{
  "mercadopago": { "accessToken": "APP_USR-…" },
  "terminals": [
    { "id": "caja-1", "name": "Caja 1 · Mercado Pago", "provider": "mercadopago",
      "deviceId": "PAX_A910__SMARTPOS1234567890" }
  ]
}
```

6 Clip · terminal

API PinPad de Clip · archivo `terminal.json`

Qué quedó integrado: el cobro (con propina) se envía al lector Clip, se consulta hasta que el cliente paga, registra marca y últimos 4 dígitos, y se puede cancelar mientras el lector no lo haya tomado.

Qué se necesita

- Cuenta de **Clip** con identidad verificada.
- Lector compatible: **Clip Total 3, Clip Ultra, Clip PinPad o Clip Stand 2**, con la app **PinPad** instalada y sesión iniciada.
- La **API de PinPad habilitada** en la cuenta y los lectores registrados.
- **Clave API** y **Clave secreta** de una aplicación de desarrollador.
- Número de serie de cada lector y WiFi estable (mínimo 10 Mb/s).

Dónde

Panel: dashboard.clip.mx → Desarrolladores. Documentación: developer.clip.mx. Soporte técnico de la API: sdk@payclip.com.

Costo

La comisión por cobro de Clip.

Paso a paso

1. Entra a dashboard.clip.mx con la cuenta del negocio y abre **Desarrolladores** (o «Credenciales API»). Crea una aplicación y copia la **Clave API** y la **Clave secreta**. La clave secreta se muestra una sola vez: guárdala en un lugar seguro.
2. Solicita a Clip que **habilite la API de PinPad** en tu cuenta y registra los lectores enviando sus números de serie (por el soporte de Clip o a sdk@payclip.com).
3. En cada lector: abre la app **PinPad**, conéctalo a la WiFi del restaurante y configura la pantalla para que no se suspenda.
4. Anota el **número de serie** (etiqueta del lector o su configuración, por ejemplo `P8220724000042`).
5. Agrega el lector a `terminal.json`:

```
{
  "clip": { "apiKey": "CLAVE_API", "secretKey": "CLAVE_SECRETA" },
  "terminals": [
    { "id": "barra", "name": "Barra · Clip", "provider": "clip", "deviceId": "P8220724000042" }
  ]
}
```

> **Nota**
>
> Si Clip acepta el cobro pero el lector no lo muestra, normalmente el lector no tiene la app PinPad activa o no quedó registrado; Clip lo confirma con su consulta de estado de lectores.

7 BBVA · terminal

Modo semi-integrado · archivo `terminal.json`

Qué quedó integrado: BBVA no publica una API para conectar sus terminales con sistemas de terceros. Por eso funciona en modo **semi-integrado**: se cobra en la terminal BBVA y en el sistema se registra el cobro con la **autorización, marca y últimos 4 dígitos** del voucher. Queda conciliado en el corte de caja.

Qué se necesita

- Contrato de **Terminal Punto de Venta (TPV)** con BBVA (ejecutivo de BBVA Empresas o Negocios).
- Nada más: no hay credenciales que obtener.

Configuración

```
{ "terminals": [
  { "id": "bbva-salon", "name": "Salón · BBVA",
    "provider": "bbva" } ] }
```

Si en el futuro BBVA entrega un SDK o API de integración (por ejemplo su producto «Total POS»), el sistema ya tiene el lugar preparado para conectarlo como Mercado Pago y Clip.

8 Stripe · ligas de pago

Stripe Checkout · archivo `onlinepay.json`

Qué quedó integrado: desde una cuenta se genera una liga de pago (pedidos por teléfono o a domicilio), se envía al cliente por WhatsApp, el sistema revisa cada 10 segundos si ya pagó y registra el pago; si la cuenta ya estaba cubierta, reembolsa. Cancelar la liga la desactiva en Stripe. No hace falta configurar webhooks.

Qué se necesita

- Cuenta de **Stripe** en México activada (datos del negocio, RFC, representante y cuenta bancaria CLABE para depósitos).
- Una **llave secreta** (`sk_live_…`) o, mejor, una **llave restringida** (`rk_live_…`).
- Una página de agradecimiento en HTTPS (a donde vuelve el cliente al pagar).

Dónde

Registro: dashboard.stripe.com/register. Llaves: dashboard.stripe.com/apikeys.

Costo

Comisión por pago con tarjeta según las tarifas vigentes de Stripe México; sin cuota mensual.

Paso a paso

1. Crea la cuenta en dashboard.stripe.com/register con el correo del negocio.
2. **Activa los pagos:** en el panel, completa «Activar pagos» con los datos de la empresa, el representante y la cuenta bancaria. Hasta entonces sólo funcionan las llaves de prueba (`sk_test_`), útiles para la primera prueba.
3. Ve a **Desarrolladores** → **Claves de API** y pulsa **Crear clave restringida**: nombre «Culinary Restaurant», permiso de **Escritura** en **Checkout Sessions** y en **Refunds** (reembolsos); lo demás sin acceso. Copia la llave.
4. Define la página de agradecimiento, por ejemplo `https://turestaurante.mx/gracias` .
5. Llena `onlinepay.json` :

```
{ "provider": "stripe", "secretKey": "rk_live_…", "successUrl": "https://turestaurante.mx/gracias" }
```

> **Recomendación**
>
> Hacer la primera prueba con la llave `sk_test_` y la tarjeta de prueba `4242 4242 4242 4242` ; después cambiar a la llave de producción.

9 WhatsApp Cloud API · mensajes

Meta · archivo `messaging.json` (sección `whatsapp`)

Qué quedó integrado: avisos automáticos al cliente (pedido en camino, listo para recoger, liga de pago), avisos al dueño y mensaje de prueba, cada uno con su **plantilla aprobada por Meta**. Si WhatsApp no está disponible, el mensaje se reintenta a los 1, 2, 4 y 8 minutos.

Qué se necesita

- **Portafolio comercial** de Meta (Business Manager), de preferencia con el negocio verificado.
- Una **app de Meta** tipo «Empresa» con el producto WhatsApp.
- Un **número de teléfono** dedicado que no esté usado en la app normal de WhatsApp.
- **Identificador del número** (phone number ID) y un **token permanente** de usuario del sistema.
- Las **5 plantillas** de mensaje aprobadas y un método de pago en WhatsApp Manager.

Dónde

Portafolio: business.facebook.com. Apps: developers.facebook.com/apps. Guía oficial: «WhatsApp Cloud API Get Started».

Costo

Meta cobra por mensaje de plantilla según la categoría (utilidad) y el país; revisar la tabla de precios de WhatsApp Business.

Paso a paso

1. En business.facebook.com crea (o usa) el **portafolio comercial** del restaurante y, en el Centro de seguridad, inicia la **verificación del negocio** (sin verificar hay límites de envío).
2. En developers.facebook.com/apps → **Crear app** → tipo **Empresa** → vincúlala al portafolio → agrega el producto **WhatsApp**.
3. En **WhatsApp** → **Configuración de la API** pulsa **Agregar número de teléfono**, verifícalo por SMS o llamada y copia el **Identificador del número de teléfono** (`phoneNumberId`).
4. Token permanente: **Configuración del negocio** → **Usuarios** → **Usuarios del sistema** → **Agregar** (rol administrador). Asígnale la app y la cuenta de WhatsApp con control total y pulsa **Generar token** con los permisos `whatsapp_business_messaging`, `whatsapp_business_management` y `business_management`. Copia el token (`accessToken`). El token temporal de la página de inicio caduca en horas: no usarlo.
5. En **WhatsApp Manager** → **Plantillas de mensajes** crea estas plantillas, categoría **Utilidad**, idioma **Español (MEX)**, con las variables en este orden exacto:
6. En WhatsApp Manager agrega un **método de pago** y llena `messaging.json`.

Plantilla (evento)	Texto sugerido	Variables en orden
`pedido_en_camino` dispatch	Hola {{1}}, tu pedido de {{2}} ya va en camino con {{3}}. Llega en unos {{4}}.	cliente, restaurante, repartidor, tiempo
`pedido_listo` pickup-ready	Hola {{1}}, tu pedido de {{2}} está listo para recoger.	cliente, restaurante

Plantilla (evento)	Texto sugerido	Variables en orden
`liga_de_pago` paylink	Hola {{1}}, paga tu cuenta de {{2}} por {{3}} aquí: {{4}}	cliente, restaurante, importe, liga
`aviso_sucursal` owner-notice	Aviso de {{1}}: {{2}}	restaurante, aviso
`mensaje_de_prueba` test	Mensaje de prueba de {{1}}.	restaurante

```
"whatsapp": {
  "phoneNumberId": "123456789012345", "accessToken": "EAAG…", "apiVersion": "v23.0", "countryCode": "52",
  "templates": {
    "dispatch": { "name": "pedido_en_camino", "language": "es_MX" },
    "pickup-ready": { "name": "pedido_listo", "language": "es_MX" },
    "paylink": { "name": "liga_de_pago", "language": "es_MX" },
    "owner-notice": { "name": "aviso_sucursal", "language": "es_MX" },
    "test": { "name": "mensaje_de_prueba", "language": "es_MX" } } }
```

10 Correo SMTP · mensajes

Buzón de envío · archivo `messaging.json` (sección `smtp`)

Qué quedó integrado: envío por correo de los avisos al dueño (límite de efectivo, arqueos, conciliaciones) y mensajes de prueba, con conexión cifrada (STARTTLS o TLS directo).

Qué se necesita

Un buzón de envío del dominio del restaurante (por ejemplo `avisos@turestaurante.mx`) con acceso SMTP autenticado: servidor, puerto, usuario y contraseña.

Según el proveedor de correo

Proveedor	Servidor y puerto	Cómo obtener la contraseña de envío
Google Workspace / Gmail	`smtp.gmail.com` · 587	Activa la verificación en dos pasos de la cuenta y crea una **contraseña de aplicación** en myaccount.google.com/apppasswords. Usa esa contraseña, no la normal.
Microsoft 365	`smtp.office365.com` · 587	En el Centro de administración de Microsoft 365 → Usuarios → el buzón → Correo → **Administrar aplicaciones de correo** → activa **SMTP autenticado**.
Hosting con cPanel	el que indique el hosting · 587 o 465	cPanel → **Cuentas de correo** → «Conectar dispositivos»: muestra servidor, puerto y usuario. Con 465 usa `"implicitTls": true`.

```
"smtp": { "host": "smtp.gmail.com", "port": 587, "username": "avisos@turestaurante.mx",
          "password": "CONTRASENA_DE_APLICACION", "from": "avisos@turestaurante.mx",
          "fromName": "Tu Restaurante", "implicitTls": false }
```

Para que no llegue a spam

Configura en el DNS del dominio los registros **SPF** y **DKIM** que indique el proveedor de correo.

11 Deliverect · Uber Eats, Rappi y DiDi Food

Integrador de plataformas · `aggregators.json` (sucursal) y `--deliverect-config` (portal)

Qué quedó integrado: Deliverect manda cada pedido de Uber Eats, Rappi o DiDi Food al portal público de la Central (firmado con HMAC); el portal lo pasa a la sucursal, donde aparece para aceptarlo o rechazarlo. Al aceptar sale a cocina con la etiqueta de la plataforma; los estados (aceptado, listo, entregado, cancelado) regresan a Deliverect y el menú se publica desde el sistema.

Qué se necesita

- Ser **socio de integración POS** de Deliverect: acuerdo de API y **certificación**.
- Credenciales **client_id** y **client_secret** (primero de staging, luego de producción) y la dirección de la API de producción.
- El restaurante con **cuenta de Deliverect** y sus canales Uber Eats, Rappi y DiDi conectados: su **accountId** y **locationId**.
- El **secreto HMAC** de la ubicación y los **ids de canal** de cada plataforma.
- La **Central publicada en internet** con HTTPS (sección 17).

Dónde

Programa de socios y documentación: developers.deliverect.com → «Become an Integration Partner».

Tiempo

La certificación se hace en llamadas con el equipo de Deliverect (en promedio dos). Las credenciales de producción llegan al aprobarla.

Paso a paso

1. En developers.deliverect.com llena el formulario **Become an Integration Partner**. El equipo de alianzas te contacta y se firma el acuerdo de API.
2. Recibirás un correo de bienvenida con la cuenta de **staging** (cuenta de socio y cuenta de cliente de prueba), el **client_id / client_secret** de staging y el documento de certificación.
3. En staging, configura como URL de pedidos del POS la del portal: `https://TU-PORTAL/webhooks/deliverect/SLUG-DEL-RESTAURANTE`. En staging el secreto HMAC es el `channelLink` o el `locationId` que indica Deliverect.
4. Prueba con la cuenta de cliente de staging: pedidos, cancelaciones, estados y publicación de menú (el PLU de cada producto es su id en el sistema). Completa las llamadas de certificación.
5. Al certificar recibirás las credenciales y la dirección de la API de **producción**, y el secreto HMAC de producción se genera en **Integration settings**.
6. El restaurante contrata Deliverect, conecta Uber Eats, Rappi y DiDi Food en su cuenta y se obtienen su **accountId** y **locationId**. Los ids de canal se consultan con el endpoint «Integrated channels» de la API.
7. Llena los dos archivos e inicia el portal con `--deliverect-config /ruta/deliverect.json`:

```
// aggregators.json (sucursal)
{ "provider": "deliverect",
  "baseUrl":
"https://api.staging.deliverect.com",
  "clientId": "…", "clientSecret": "…",
  "accountId": "…", "locationId": "…" }
```

```
// deliverect.json (portal público)
{ "slug-del-restaurante": {
    "secret": "SECRETO_HMAC",
    "channels": { "ID_UBER": "uber",
      "ID_RAPPI": "rappi", "ID_DIDI": "didi" } }
}
```

12 Cloudbeds · cargo a habitación

API v1.3 · archivo `pms.json` con `"system": "cloudbeds"`

Qué quedó integrado: el mesero indica habitación y apellido; el sistema busca la reservación hospedada (checked-in), valida el apellido y el crédito semanal, carga el consumo al folio en Cloudbeds y registra el pago a habitación. Un cargo nunca se duplica aunque se repita.

Qué se necesita

- Que el hotel use **Cloudbeds** y un usuario **propietario o administrador** de la propiedad.
- Una **llave de API** (empieza con `cbat_`) con permisos de lectura de reservaciones y escritura de artículos.
- El **Property ID** del hotel.

Dónde

En Cloudbeds: **Account → Apps & Marketplace → API Credentials**. Documentación: developers.cloudbeds.com.

Costo

Sin costo adicional para la propiedad (API de autoservicio).

Paso a paso

1. El hotel entra a Cloudbeds con un usuario propietario o administrador y abre **Account → Apps & Marketplace → API Credentials**.
2. Pulsa **+ New Credentials**: nombre «Culinary Restaurant»; en Redirect URI se puede poner `https://localhost` (no se usa con llave de API).
3. Genera la **API key** con los permisos `read:reservation` y `write:item` y cópiala: sólo se muestra una vez. Deja de funcionar si pasa 30 días sin usarse.
4. Toma el **Property ID** de la configuración de la propiedad.
5. Los meseros escriben la habitación con el mismo nombre que tiene en Cloudbeds.

```
{ "system": "cloudbeds", "creditLimit": 5000, "itemName": "Consumo en restaurante",
  "cloudbeds": { "apiKey": "cbat_…", "propertyId": "123456" } }
```

`creditLimit` es el crédito por habitación en la semana, en pesos (0 = lo decide el hotel).

13 Oracle OPERA Cloud · cargo a habitación

Oracle Hospitality Integration Platform (OHIP) · archivo `pms.json` con `"system": "opera"`

Qué quedó integrado: lo mismo que con Cloudbeds, sobre las API de OHIP: busca la reservación «In House» de la habitación, valida apellido y crédito y publica el cargo en el folio con el código de transacción del restaurante. Si se pierde la respuesta, busca el cargo en el folio antes de repetirlo.

Qué se necesita

- Hotel con **OPERA Cloud** y suscripción a **OHIP**.
- Un usuario de OPERA con el rol **DEVELOPERPORTALACCESS** (lo asigna el administrador del hotel).
- **Application Key** de una aplicación registrada en el Developer Portal.
- **Gateway URL, Client ID, Client Secret y Enterprise ID** del ambiente.
- **Hotel ID** (código de la propiedad), el **Transaction Code** para consumos del restaurante y un **Cashier ID**.

Dónde

Oracle Hospitality Developer Portal (enlace dentro del menú de OPERA Cloud). Hay portales separados para **UAT** (pruebas) y **PROD**. Especificaciones: github.com/oracle/hospitality-api-docs.

Costo

Según el contrato de OHIP del hotel con Oracle.

Paso a paso

1. El administrador del hotel asigna el rol **DEVELOPERPORTALACCESS** al usuario que hará la configuración.
2. Desde OPERA Cloud abre el **Developer Portal** (primero el de UAT). En **Applications** → **Register Application**: paso 1 nombre y contacto; paso 2 suscribe **Property APIs**. En «View Details» copia la **Application Key**.
3. En **Environments** → **View Details** copia **Gateway URL, Client ID, Client Secret y Enterprise ID** (si no hay Client ID, pulsa **Create Client Credentials**). Si no aparecen credenciales, Oracle aún no termina de habilitar el ambiente para la propiedad.
4. Pide al hotel el **Hotel ID** exacto (respeta mayúsculas), el **Transaction Code** de restaurante y el **Cashier ID**.
5. Para producción repite el registro en el portal **PROD**: es una aplicación y una llave distintas.

```
{ "system": "opera", "creditLimit": 5000,
  "opera": { "gatewayUrl": "https://GATEWAY", "tokenUrl": "https://GATEWAY/oauth/v1/tokens",
    "grantType": "client_credentials", "clientId": "…", "clientSecret": "…",
    "scope": "urn:opc:hgbu:ws:__myscopes__", "enterpriseId": "…", "appKey": "…",
    "hotelId": "HOTEL1", "transactionCode": "2000", "cashierId": 1 } }
```

14 Google Maps · ubicación y distancia

Geocoding API y Routes API · archivo `maps.json`

Qué quedó integrado: al guardar la dirección de un cliente con «Ubicar», Google calcula sus coordenadas y la ruta desde el restaurante (distancia y tiempo, más 5 minutos de entrega). La entrega a domicilio toma esos datos.

Qué se necesita

- Proyecto de **Google Cloud** con **facturación** activa (tarjeta).
- Las APIs **Geocoding API** y **Routes API** habilitadas.
- Una **clave de API restringida**.

Dónde

console.cloud.google.com.

Costo

Google cobra por solicitud después de un uso mensual gratuito; un restaurante normalmente queda dentro o cerca de ese rango. Revisar la tabla de precios de Google Maps Platform.

Paso a paso

1. En console.cloud.google.com crea un proyecto («Culinary Restaurant»).
2. En **Facturación** vincula una cuenta de facturación.
3. En **APIs y servicios** → **Biblioteca** habilita **Geocoding API** y **Routes API**.
4. En **APIs y servicios** → **Credenciales** → **Crear credenciales** → **Clave de API**. Luego **Editar clave**: en «Restricciones de API» permite sólo Geocoding y Routes; en «Restricciones de aplicación» elige **Direcciones IP** y pon la IP pública del servidor.
5. En la tarjeta **Mapas** de Integraciones configura la latitud y longitud del restaurante (punto de partida de las rutas).

```
{ "apiKey": "AIza…", "region": "mx", "language": "es", "travelMode": "TWO_WHEELER" }
```

`travelMode`: `TWO_WHEELER` para repartidores en moto o `DRIVE` para auto.

15 Asterisk · identificador de llamadas

AMI sobre TLS · archivo `callerid.json`

Qué quedó integrado: el servidor se conecta al conmutador, muestra la llamada entrante en Delivery y la liga al cliente por los últimos 10 dígitos; se reconecta solo si se cae la conexión.

Qué se necesita

Conmutador **Asterisk** (o FreePBX) en la red del restaurante con **AMI sobre TLS**, un certificado válido para su nombre que el servidor reconozca, y un **usuario AMI de sólo lectura**. Lo configura quien administre el conmutador.

```
; /etc/asterisk/manager.conf
[general]
enabled = yes
tlsenable = yes
tlsbindaddr = 0.0.0.0:5039
tlscertfile = /etc/asterisk/keys/asterisk.crt
tlsprivatekey = /etc/asterisk/keys/asterisk.key

[culinary-readonly]
secret = UNA_CONTRASENA_LARGA
read = call
write =
deny = 0.0.0.0/0.0.0.0
permit = IP_DEL_SERVIDOR/255.255.255.255
```

Después: `asterisk -rx "manager reload"`. En FreePBX se crea en **Configuración → Usuarios de Asterisk Manager**. Si el certificado lo firma una CA propia, instálala en el servidor Debian (`/usr/local/share/ca-certificates` y `update-ca-certificates`).

```
{ "host": "conmutador.local", "port": 5039, "username": "culinary-readonly", "secret":
"UNA_CONTRASENA_LARGA" }
```

16 Impresora de tickets y cajón

ESC/POS · archivo `printer.json`

Qué quedó integrado: impresión de cuentas y cortes Z y apertura del cajón al cobrar en efectivo; revisa papel y tapa antes de imprimir y nunca reimprime solo un ticket interrumpido.

- Impresora térmica **ESC/POS** en red (puerto RAW 9100, con **IP fija**) o USB.
- Cajón conectado al puerto **RJ11** de la impresora.
- USB: agregar el usuario del sistema al grupo de impresoras: `sudo usermod -aG lp culinaryos-app`.

```
{ "uri": "tcp://10.77.0.20:9100", "columns": 48 }     // 48 = papel de 80 mm, 32 = 58 mm
{ "uri": "file:///dev/usb/lp0", "columns": 32 }       // impresora USB
```

17. Central publicada en internet (necesaria para Deliverect)

Deliverect necesita entregar los pedidos a una dirección pública. La sucursal no está expuesta a internet; los recibe el **portal público de la Central**, que hoy escucha sólo en el propio servidor.

1. Un servidor Linux en la nube (VPS) con IP pública donde corran la Central y el portal.
2. Un dominio o subdominio, por ejemplo `pedidos.turestaurante.mx`, apuntando a esa IP.
3. Certificado TLS (por ejemplo Let's Encrypt) y un proxy inverso (nginx o Caddy) que reenvíe HTTPS al portal (puerto local 9445).
4. Iniciar el portal con `--deliverect-config` y dar a Deliverect la URL `https://pedidos.turestaurante.mx/webhooks/deliverect/SLUG`.
5. Conectar cada sucursal a la Central (opción `--cloud-url` del servidor de la sucursal) con sus certificados mTLS.

18. Lista de verificación para la prueba final

Orden sugerido: primero los ambientes de prueba de cada proveedor y después producción.

Integración	Prueba mínima con la cuenta real	Hecho
Facturama	Timbrar una factura en sandbox, revisarla con el contador, cancelarla; luego una real en producción.	☐
Mercado Pago Point	Cobro aprobado, cobro cancelado desde el sistema y devolución.	☐
Clip	Cobro aprobado con propina y cobro rechazado.	☐
BBVA	Registrar un cobro con el voucher y verlo en el corte.	☐
Stripe	Liga pagada con tarjeta de prueba, liga cancelada; después un pago real pequeño.	☐
WhatsApp	Mensaje de prueba y aviso «pedido en camino» a un número propio.	☐
Correo	Mensaje de prueba al correo del dueño (revisar que no llegue a spam).	☐
Deliverect	Pedido de staging aceptado, listo y entregado; menú publicado; cancelación.	☐
Cloudbeds / OPERA	Cargo a una habitación de prueba y apellido equivocado rechazado.	☐
Google Maps	Guardar una dirección real y revisar distancia y tiempo.	☐
Asterisk	Llamar al restaurante y ver la llamada entrante con la ficha del cliente.	☐
Impresora	Imprimir una cuenta y abrir el cajón con un cobro en efectivo.	☐

19. Puntos por confirmar en la prueba real

Estos detalles no se pueden comprobar sin una cuenta real y se revisan durante la prueba; si algo difiere, el ajuste es pequeño.

- **Deliverect:** el formato exacto de publicación del menú y los ids de canal de Uber Eats, Rappi y DiDi Food; la dirección de la API de producción la da Deliverect al certificar.
- **Mercado Pago:** si la respuesta de la orden incluye los últimos 4 dígitos de la tarjeta (hoy se registra la marca).
- **Clip:** que la API de PinPad quede habilitada para los lectores del restaurante.
- **OPERA (OHIP):** el endpoint y el scope de autenticación del ambiente de cada hotel.
- **WhatsApp:** la aprobación de las plantillas por Meta (puede tardar de minutos a un día).

> **Dónde está el detalle técnico**
> Código y descripción completa en el PR #4 (github.com/errPizza/CulinaryRestaurant/pull/4). Documentación en el repositorio: `docs/PROVEEDORES.md`, `docs/CULINARYOS.md` (sección «Integraciones reales») y los ejemplos de `deploy/integrations/`.